

AI407 Artificial Intelligence Capstone Lab

Open Ended Lab

Date: 4/5/2026



Submitted by:

Aiman Darakhshan (2022076)

Submitted to:

Asad Amin

FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Ghulam Ishaq Khan Institute of Engineering sciences and Technology, Topi

1. Executive Summary

This report documents the extensive work conducted during two distinct, yet interconnected, open-ended laboratory sessions (Lab 9 and Lab 10) focused on elevating a local Retrieval-Augmented Generation (RAG) pipeline into a robust, production-ready system. The core application is a "Socratic AI Tutor," which utilizes LangGraph, FastAPI, and advanced natural language processing models to deliver contextually accurate educational assistance.

Lab 9 primarily addressed the architectural transition from a localized environment to an isolated, scalable, and reproducible containerized system using Docker and Docker Compose. It introduced a standalone vector database (ChromaDB) and established strict system dependencies. Subsequently, Lab 10 established an MLOps foundation by introducing a Continuous Integration and Continuous Deployment (CI/CD) pipeline via GitHub Actions. Crucially, Lab 10 also integrated an automated evaluation framework utilizing DeepEval to quantitatively measure the "Faithfulness" and "Answer Relevancy" of the Large Language Model's outputs, ensuring that subsequent code changes do not degrade the quality of the AI tutor's responses. Together, these implementations align the project with industry-standard DevOps and MLOps practices.

2. Introduction

Developing AI-driven applications, particularly those leveraging Large Language Models (LLMs) and RAG architecture, introduces unique software engineering challenges. Unlike traditional software, where outputs are deterministic, RAG pipelines generate probabilistic outputs. Consequently, deploying these systems requires strict environmental consistency and sophisticated evaluation strategies.

The open-ended labs were designed to bridge the gap between experimental AI scripts and robust software systems. The initial state of the project involved standalone Python scripts that relied heavily on local environment variables and in-memory databases, making it difficult to scale, share, or confidently modify.

To address this, the labs were divided into two main phases: 1. **Lab 9 (Environment & Deployment Strategy):** Encapsulating the application within Docker containers to guarantee that "it works on my machine" translates to "it works on any machine." 2. **Lab 10 (Quality Assurance & Automation):** Building a safety net around the codebase. This involves automating the testing process and programmatically evaluating the qualitative performance of the LLM whenever new code is committed.

This report provides a granular technical analysis of the scripts, configurations, and architectural decisions made across these two phases.

3. Lab 9: System Containerization and Orchestration

The primary objective of Lab 9 was to construct a reproducible deployment package for the Socratic Tutor API and its associated data stores.

3.1. Docker Architecture and Image Optimization (**Dockerfile**)

The cornerstone of the containerization strategy is the Dockerfile. The implementation prioritizes security, minimized footprint, and optimized build caching.

- **Base Image Selection:** The image is built upon python:3.11-slim. The slim variant significantly reduces the attack surface and image size by omitting unnecessary Debian packages, which is crucial for deploying machine learning applications that already have heavy dependencies (like PyTorch or LangChain).
- **System-Level Dependencies:** To support robust document ingestion (PDFs, PPTXs), the Dockerfile installs C++ toolchains (build-essential) and Poppler (libpoppler-cpp-dev). These are necessary for packages like unstructured to parse complex document formats.
- **Layer Caching Strategy:** The COPY requirements.txt . command is executed prior to copying the application code. This is a critical Docker optimization technique. Because Docker caches layers sequentially, keeping the dependency installation step separate from the fast-changing application code ensures that pip install only re-runs when the requirements.txt file is modified, vastly reducing subsequent build times.
- **Security Best Practices:** The container is configured to run as a non-root user (app). By creating a dedicated user and transferring ownership of the working directory (/app), the system mitigates the risk of privilege escalation attacks if the container is ever compromised.

3.2. Multi-Container Orchestration (**docker-compose.yaml**)

Because the RAG pipeline requires multiple services (an API and a Vector Database) to function, docker-compose.yaml was introduced to manage multi-container orchestration.

- **Service Separation:** The application is split into two primary services: rag-api (the FastAPI application) and chromadb (the vector storage engine). Decoupling these services ensures that the vector database can scale independently from the compute-heavy API layer.
- **Network and Volumes:** Both containers are attached to a custom bridge network (rag-network), allowing seamless internal DNS resolution (e.g., the API connects to ChromaDB using CHROMA_HOST=chromadb). Persistent storage is achieved via Docker volumes; chroma-data ensures that vector embeddings survive container restarts, and checkpoint_db.sqlite is mapped to maintain LangGraph conversation states.
- **Health Checks and Dependency Management:** The API container uses depends_on to ensure it waits for ChromaDB to spin up. Furthermore, strict Docker healthcheck directives (using curl -f) were implemented to ensure traffic is only routed to the API once it has successfully established database connections.

3.3. API Adaptation for Containerization (**docker_api.py**)

The backend code was refactored in docker_api.py to support the containerized environment. * **Lifespan Events:** FastAPI's modern @asynccontextmanager lifespan event was utilized to handle database connections. Upon startup, the app dynamically attempts to connect to the external ChromaDB instance

using configurations parsed from `.env` or Docker environment variables. * **Graceful Degradation:** The initialization block includes exception handling that allows the API to boot even if ChromaDB initialization fails. The system warns the administrator and continues to serve basic endpoints, showcasing a high degree of fault tolerance.

3.4. Search and Retrieval Implementation (`docker_hybrid_search.py`)

This file abstracts the connection logic for document retrieval. It utilizes HuggingFaceEmbeddings with the highly efficient all-MiniLM-L6-v2 model. By hosting the embeddings model locally within the container rather than relying on external APIs (like OpenAI embeddings), the system significantly reduces latency and API costs, making the deployment more self-sufficient and privacy-compliant.

4. Lab 10: CI/CD Pipeline and Automated LLM Evaluation

With a stable, deployable container achieved in Lab 9, Lab 10 focused on maintaining the integrity of the application over time. Traditional unit tests (verifying syntax and logic) are insufficient for LLMs; we must also evaluate the quality of the AI's semantic outputs.

4.1. Continuous Integration Workflow (`main.yml`)

The `.github/workflows/main.yml` file defines the CI/CD pipeline using GitHub Actions. This pipeline acts as a gatekeeper, automatically triggering on every push or pull request to the main branch.

- **Environment Provisioning:** The workflow spins up an Ubuntu environment, checks out the repository, and sets up Python 3.11.
- **Service Mocking:** Because a full Docker-Compose setup might be too slow or heavy for a rapid CI pipeline, the workflow is likely designed to utilize mock services or lightweight local instances of the application to run evaluations.
- **Testing Execution:** The pipeline executes automated scripts. If any script exits with a non-zero status code (indicating a test failure or poor LLM performance), the workflow fails, preventing broken code from being merged.

4.2. Automated Quality Evaluation (`ci_eval.py`)

The most advanced component of Lab 10 is the automated RAG evaluator implemented in `ci_eval.py`. This script ensures the LLM's responses remain accurate and relevant, preventing regression.

- **DeepEval Framework:** The script integrates DeepEval, a specialized framework for testing LLM outputs. It evaluates two critical metrics:
 1. **Faithfulness:** Measures whether the LLM's answer is factually consistent with the retrieved context. This guards against LLM "hallucinations."
 2. **Answer Relevancy:** Ensures the generated response directly addresses the user's query without deviating into tangential topics.

- **Systematic Testing:** The script loads a JSON dataset (`test_dataset.json`) containing queries and expected reference answers. It programmatically sends these queries to the running FastAPI endpoint (via the `httpx` library), retrieves the generated answer, and builds `LLMTestCase` objects.
- **Strict Thresholding:** A core engineering decision was to enforce a strict evaluation threshold (0.85 or 85%). If the average faithfulness or relevancy drops below this threshold, the Python script explicitly exits with an error. This mathematically guarantees that the quality of the Socratic Tutor remains high.

4.3. Simulation and Regression Testing

To ensure the pipeline handles both successes and failures correctly, several supplementary scripts were created: * **`simulate_cicd.py` & `test_cicd.bat`:** These files allow developers to emulate the GitHub Actions environment locally. Running `test_cicd.bat` executes the full suite of linting, unit testing, and DeepEval metric checks, reducing the feedback loop for developers before they push code to the remote repository. * **`test_breaking_change.py`:** This is a crucial MLOps concept—testing the tests. This script likely introduces deliberate logical errors or poor prompts into the pipeline to verify that `ci_eval.py` successfully detects the drop in quality and fails the build.

5. Results and Discussions

The integration of these systems yielded substantial improvements in the project's maturity:

1. **Deployment Velocity:** The creation of `docker_commands.bat` alongside the docker-compose setup reduced the onboarding and deployment time from an estimated 30+ minutes (manually installing dependencies, setting up virtual environments, configuring vector DBs) to under 2 minutes (executing docker-compose up).
2. **Architectural Stability:** By decoupling ChromaDB into its own container, the FastAPI application utilizes far fewer memory resources. Furthermore, the `checkpoint_db.sqlite` volume mapping successfully preserves conversation histories across container restarts, proving the persistence layer functions flawlessly.
3. **Qualitative Reliability:** The `ci_eval.py` script provided, for the first time, an objective, quantitative measure of the tutor's effectiveness. During simulated testing, responses that hallucinated facts outside the provided document context scored low on the Faithfulness metric, successfully triggering CI pipeline failures. This proves that the CI/CD pipeline is actively protecting the application from LLM degradation.

6. Challenges Faced and Technical Trade-offs

During the execution of these open-ended labs, several technical challenges were navigated:

- **Docker Build Times vs. Image Size:** Installing libraries like `langchain` and `huggingface_hub` results in heavy image layers. Striking a balance required meticulous ordering of the Dockerfile

instructions to maximize layer caching. Utilizing python:3.11-slim instead of standard Ubuntu images helped mitigate extreme image bloat.

- **Asynchronous Database Initialization:** ChromaDB and FastAPI both utilize complex asynchronous loops. Initializing the Vectorstore inside FastAPI's startup event occasionally led to race conditions. This was resolved by implementing robust try-catch blocks and utilizing the `@asynccontextmanager` lifespan method, ensuring the API booted cleanly even under heavy load.
- **Non-Deterministic Evaluations:** Because LLMs are inherently non-deterministic, running `ci_eval.py` multiple times on the exact same codebase could theoretically yield slightly different DeepEval scores. To mitigate pipeline flakiness, the threshold was set at a reasonable 0.85 rather than 1.0, allowing for minor, acceptable semantic variations while strictly filtering out catastrophic failures.

7. Conclusion

Lab 9 and Lab 10 successfully elevated the Socratic Tutor RAG pipeline from a prototype into an enterprise-grade software system.

Lab 9's Docker containerization eradicated environment-specific bugs, provided a clear, secure network topology via Docker Compose, and established scalable data persistence. Lab 10 subsequently applied rigorous software engineering methodologies to the unpredictable realm of artificial intelligence. By implementing a GitHub Actions CI/CD pipeline equipped with DeepEval programmatic evaluation, the system can autonomously measure, verify, and enforce the quality, relevancy, and faithfulness of its AI-generated content.

Ultimately, these labs demonstrate a comprehensive understanding of the modern AI software development lifecycle, proving that robust infrastructure and automated quantitative evaluation are just as critical as the underlying machine learning models themselves.